

N044 사람이 승인하는 에이전트 만들기_학습서

Chapter01

사람이 승인하는 에이전트 만들기

지금까지 만든 자동화는 자료를 모으고 분류하고 조사해서 결과물을 만들어 내는 데까지였습니다. 그런데 그 결과물을 실제로 고객에게 보내거나 시스템에 반영해야 하는 순간이 오면, AI가 내린 판단을 그대로 믿고 내보내도 되는지 한 번쯤 망설이게 됩니다. 이번 시간에는 자동화의 마지막 단계에 사람이 확인하고 승인하는 과정을 넣어서, 결과를 안심하고 내보낼 수 있는 자동화를 완성해 보겠습니다.

정확도 향상만으로 풀리지 않는 문제

환불을 집행하거나 메일을 발송하고, 문서를 고치거나 문의를 종결 처리하는 일은 한 번 실행되고 나면 되돌리기가 어렵습니다. AI의 판단이 아무리 정확해져도 100%에 이르지 않는 기 때문에, 이런 일을 AI에게 맡기면 틀린 결과가 그대로 바깥으로 나가는 경우가 생길 수밖에 없습니다.

숫자로 따져 보면, 정확도가 95%인 AI에게 하루 200건을 맡겼을 때 매일 10건 정도가 확인 없이 잘못 나가고 정확도를 99%까지 끌어올려도 하루 2건은 남습니다. 그 2건이 31만 원짜리 환불이거나 고객에게 잘못 전달된 안내라면, 정확도가 올라간 것과 상관없이 그 건의 손해는 그대로 남습니다.

이럴 때 흔히 프롬프트를 더 다듬어 정확도를 높이려고 하지만, 그렇게 해도 남은 2건이 0건이 되지는 않습니다. 그래서 판단을 더 정확하게 만드는 노력과는 별개로, AI가 틀렸을 때 결과가 나가기 전에 막을 수 있는 자리를 따로 마련해 두어야 합니다.

세 가지 선택지

AI에게 되돌리기 어려운 일을 맡길 때 고를 수 있는 방법은 크게 세 가지입니다.

첫 번째는 AI가 처리한 결과를 사람이 모두 확인하는 방법입니다. 가장 안전하지만 200건이 들어오면 사람이 200번을 봐야 하니, AI를 쓰기 전과 비교해 줄어드는 일이 거의 없습니다. 두 번째는 반대로 모든 결과를 자동으로 내보내는 방법인데, 처리 속도는 빠르지만 잘못된 결과가 나가도 막을 방법이 없어서 사고를 그대로 감수해야 합니다.

세 번째는 위험한 건에서만 AI가 멈추고 사람에게 확인을 받은 뒤 이어서 처리하는 방법입니다. 200건 중 위험한 20건만 사람에게 가고 나머지 180건은 자동으로 끝나기 때문에, 자동화의 속도를 유지하면서도 큰 사고는 사람이 막을 수 있습니다. 이 방식을 Human-in-the-Loop, 줄여서 HITL이라고 부릅니다.

HITL을 만들려면 두 가지를 준비해야 합니다. 하나는 일을 하던 중에 멈췄다가 사람의 답을 받아 이어서 실행하는 기능이고, 다른 하나는 어떤 건에서 멈출지 정하는 기준입니다. 둘 다 에이전트의 구조를 짤 때 처음부터 함께 정해 두어야 하는 것이어서, 이번 시간에는 멈추고 이어가는 기능이 어떻게 동작하는지를 중심으로 살펴보고 멈춤 기준은 왜 검증해야 하는지 까지 함께 다룹니다.

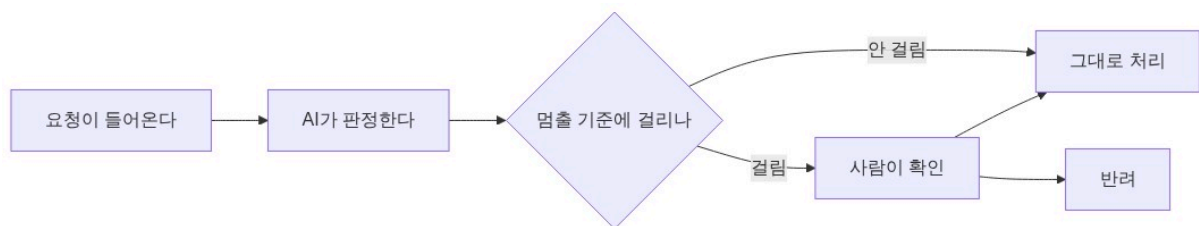


그림: 위험한 건만 사람에게 보내고 나머지는 자동으로 처리한다

HITL로 만들 수 있는 자동화

멈췄다 이어가는 구조를 갖추면 아래와 같은 자동화를 만들 수 있습니다. 모두 결과가 바깥으로 나가고 한 번 나가면 되돌리기 어려우며, 건마다 위험의 크기가 달라서 일부만 사람이 확인하면 되는 일들입니다.

자동화하려는 일	되돌릴 수 없는 것	사람이 봐야 할 만한 건
환불과 보상 집행	돈이 나간다	금액이 크거나 규정 해석이 갈리는 건
고객 답변 자동 발송	고객이 읽는다	AI가 확신하지 못한 분류, 규정 인용이 불확실한 답변
콘텐츠 자동 게시	구독자가 읽는다	출처가 불분명한 수치가 들어간 글
문서 자동 수정 반영	원본이 바뀐다	고친 분량이 큰 건
문의 자동 종결	요청자의 요청이 닫힌다	아직 해결되지 않은 신호가 남은 건
일정 자동 확정	상대방 시간이 잡힌다	일정이 겹치거나 요청이 모호한 건

이번 시간에는 이 중 환불 집행을 예로 들어, 에이전트가 멈췄다가 이어서 실행되는 과정을 직접 확인하고 멈춤 기준을 숫자로 검증해 보겠습니다.

Chapter02

HITL을 두는 자리

업무를 단계로 나눠 보면 앞쪽 단계는 대부분 몇 번을 다시 해도 괜찮습니다. 요청을 읽고, 규정을 찾고, 환불 여부를 정하는 일은 잘못되면 다시 하면 됩니다. 되돌릴 수 없게 되는 것은 **결과가 바깥으로 나가는 단계**입니다. 송금, 메일 발송, 게시, 외부 시스템 등록이 여기 해당합니다. HITL은 이 단계 바로 앞에 둡니다. 만드는 일과 보내는 일을 서로 다른 단계로 나눠 두면 그 사이가 자연스럽게 HITL의 자리가 됩니다.

멈추고 이어가는 방식

자리가 정해지면 그 자리에서 실행을 멈추는 기능이 필요합니다. 어디서 멈추고, 멈춘 동안 무엇을 저장하고, 어떻게 이어갈지가 이 에이전트 구조의 뼈대입니다. 동작 원리를 알아야 이 뼈대를 제대로 짤 수 있습니다.

에이전트는 여러 단계를 순서대로 실행합니다. 환불이라면 판정하고, 확인하고, 집행합니다. HITL은 이 흐름의 중간, 집행 바로 앞에서 실행을 멈추고 사람의 답을 기다렸다가, 답이 오면 **처음부터가 아니라 멈춘 지점부터** 나머지 단계를 이어서 실행하는 구조입니다. 여기에 쓰이는 기능이 둘입니다.

interrupt: 실행 멈추기

에이전트가 일을 하다가 `interrupt` 를 만나면 **그 자리에서 멈춥니다**. 그리고 `interrupt` 에 담은 내용이 담당자에게 전달됩니다. 여기에 무엇을 담을지가 곧 승인 화면의 내용이 됩니다.

```
answer = interrupt({
  "요청": "코트를 한 달 입었는데 어깨 솔기가 터졌습니다.",
  "금액": "310,000원",
  "통과시키면": "310,000원이 계좌로 나갑니다",
})
```

멈춘 시점의 진행 상황은 **체크포인트**라는 부품에 저장됩니다. 체크포인트는 이 섹션 뒤쪽에서 따로 설명합니다.

resume: 담당자 답변으로 이어가기

담당자가 답을 주면 `Command(resume=답)` 으로 에이전트에 넣습니다. 그러면 멈춰 있던 자리부터 이어지고, 위 코드의 `answer` 에 담당자의 답이 들어옵니다. 에이전트는 그 답을 보고 다음 단계를 고릅니다. 승인이면 집행으로, 반려면 반려 처리로 갑니다.

어느 건을 이어갈지는 `thread_id` 로 구분합니다. 요청 한 건마다 고유한 값을 붙이므로 대기 중인 건이 여러 개여도 섞이지 않습니다.

체크포인트: 멈춘 상태 저장하기

`interrupt` 로 멈추면 에이전트의 실행은 그 자리에서 끝납니다. 프로그램 입장에서는 함수 호출이 한 번 반환된 것입니다. 그런데 나중에 `resume` 으로 이어가려면 두 가지를 알아야 합니다. 지금까지 어떤 값이 계산되어 있었는지, 그리고 다음에 어느 단계를 실행해야 하는지입니다. 이 정보를 저장해 두는 부품이 **체크포인트**입니다.

체크포인트는 에이전트가 단계 하나를 마칠 때마다 그 시점의 상태를 기록합니다. 기록 한 건을 **체크포인트**라고 부르고, 여기에는 두 가지가 들어갑니다. 그 시점까지 쌓인 값(요청 내용, 판정 결과 등)과 다음에 실행할 단계의 이름입니다. `resume` 이 들어오면 에이전트는 가장 최근 체크포인트를 꺼내 값을 복원하고, 적혀 있는 다음 단계부터 실행합니다. 처음부터 다시 돌지 않고 멈춘 자리에서 이어지는 이유가 이것입니다.

체크포인트는 `thread_id` 별로 따로 쌓입니다. 요청 R3과 R4가 동시에 멈춰 있어도 각자의 기록이 분리되어 있어 섞이지 않습니다. 같은 `thread_id` 로 부르면 그 요청의 기록에서 이어가고, 새 `thread_id` 로 부르면 빈 기록에서 새로 시작합니다.

체크포인트가 없으면 `interrupt` 를 쓸 수 없습니다. 멈춘 뒤 이어갈 근거가 남지 않기 때문입니다. 그래서 에이전트를 만들 때 체크포인트를 함께 지정합니다. 아래 실습 코드에서는 이 한 줄입니다.

```
app = g.compile(checkpointer=InMemorySaver())
```

체크포인트는 기록을 어디에 두느냐에 따라 종류가 나뉩니다.

체크포인트	저장 위치	프로그램을 꺾다 켜면
<code>InMemorySaver</code>	메모리	기록이 사라진다
<code>SqliteSaver</code>	파일 하나	기록이 남는다
<code>PostgresSaver</code>	데이터베이스	기록이 남고, 여러 서버가 함께 쓸 수 있다

오늘 실습은 `InMemorySaver` 로 충분합니다. 실제 서비스에서 사람의 답을 몇 시간씩 기다려야 한다면 파일이나 데이터베이스에 저장하는 쪽을 써야 하고, 이 문제는 섹션 4에서 다시 다룹니다.

실습을 시작하기 전에 필요한 패키지를 설치합니다.

실행 되었습니다.

실행

직접 확인하기

아래 셀은 31만 원짜리 R3 요청 하나를 넣고, 멈춘 뒤 담당자의 답을 넣어 이어가는 과정을 보여 줍니다. 코드는 읽지 않아도 됩니다. 실행한 뒤 출력을 보고, **ANSWER** 를 **"approve"** 로 바꿔 한 번 더 실행해 보세요.

이 노트북의 코드 셀은 멈추고 이어가는 동작 자체를 보여 주기 위한 예시라서, 실제 에이전트라면 AI가 맡았을 판단 단계를 넣지 않았습니다. 요청을 받으면 LLM이 환불 여부를 판단하는 대신 바로 멈추거나 정해 둔 규칙대로 처리하기 때문에, API 키 없이 실행되고 몇 번을 돌려도 같은 결과가 나옵니다. 실제 프로젝트에서는 이 자리에 LLM의 판단을 넣어 완성합니다.

실행 되었습니다.

실습: 멈췄다가 담당자 답변으로 이어가기 (ANSWER만 수정)

실행

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45

46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65

결과 해석: 멈춘 뒤 반려로 이어짐

①에서 에이전트는 돈을 보내기 전에 멈췄습니다. "돈이 나갔는가: False" 가 그 증거입니다. 이때 담당자에게 보이는 것은 `interrupt` 에 담은 세 줄입니다.

②에서 담당자가 `reject` 를 넣자 멈춘 자리부터 이어져 반려 처리로 갔고, 0원이 나갔습니다. `ANSWER` 를 `"approve"` 로 바꿔 실행하면 같은 자리에서 집행으로 가서 `310,000원 환불 집행` 이 찍힙니다. 같은 에이전트인데 담당자의 한 단어로 길이 갈립니다.

체크포인트에 저장된 내용 확인하기

체크포인트가 실제로 무엇을 저장했는지 꺼내 볼 수 있습니다. `get_state` 는 특정 `thread_id` 의 가장 최근 체크포인트를 돌려주고, `get_state_history` 는 지금까지 쌓인 체크포인트 전체를 돌려줍니다. 아래 셀은 R3을 새 `thread_id` 로 한 번 더 멈춰 세운 뒤 저장 내용을 보고, 재개해서 끝낸 뒤 다시 봅니다. 바꿀 값은 없으니 실행만 하세요.

실행 되었습니다.

실습: 체크포인트에 저장된 내용 보기

실행

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19

결과 해석: 멈춘 직후와 끝난 뒤의 체크포인트

①에서 저장된 값은 `req` 하나뿐입니다. 요청 내용만 들어 있고, 담당자의 답(`decision`)과 집행 금액(`paid`)은 아직 없습니다. 다음에 실행할 단계에는 `review` 가 적혀 있습니다. 멈춘

단계가 **review** 였으니, 재개하면 여기서부터 실행하겠다는 기록입니다.

바로 아래에서 다들 "멈춘 단계가 처음부터 다시 실행된다" 는 동작도 이 기록에서 나오니다. 체크포인트에는 **review** 안의 몇 번째 줄까지 실행했는지가 아니라 **review** 라는 단계 이름만 적혀 있습니다. 그래서 이어갈 때 **review** 를 첫 줄부터 실행합니다.

②에서는 **decision** 과 **paid** 가 추가됐고, 다음에 실행할 단계는 빈 값 **()** 입니다. 더 실행할 단계가 없으니 이 요청은 처리가 끝났다는 뜻입니다. 체크포인트는 2개에서 4개로 늘었습니다. **review** 와 **execute** 가 끝날 때마다 하나씩 기록되었습니다. 중간의 **→ 310,000원 환불 집행** 은 재개할 때 **execute** 단계가 찍은 줄입니다.

다음 단계가 남아 있으면 처리 중이거나 대기 중이고, 비어 있으면 끝난 것입니다. 승인 대기 목록을 만들 때 이 값을 기준으로 대기 중인 건을 골라냅니다.

주의: 멈춘 단계의 재실행

이어갈 때 주의할 동작이 하나 있습니다. 에이전트는 **interrupt** 가 있던 줄의 다음 줄부터 이어가지 않고, **멈췄던 단계를 처음부터 한 번 더 실행**합니다. 실행 중이던 함수의 중간 상태는 저장할 수 없어서 이렇게 설계되어 있습니다.

이것이 실무에서 사고로 이어집니다. 예를 들어 멈추기 직전에 "고객님 요청을 검토 중입니다" 문자를 보내도록 만들어 두었다면, 이어갈 때 그 단계가 다시 실행되어 **고객이 같은 문자를 두 번 받습니다**.

그래서 구조를 짤 때 이 원칙을 지켜야 합니다. **"사람에게 묻는 단계 안에서, 묻기 전에 바깥으로 나가는 동작을 두지 말 것."** 문자 발송이나 알림은 멈추는 단계와 분리해 따로 둡니다.

Chapter03

멈춤 기준 정하기

멈추는 방법을 알았으면 다음은 **언제 멈출지**입니다. 모든 건에서 멈추면 사람이 전부 처리하는 것과 같고, 하나도 멈추지 않으면 HITL이 없는 것과 같습니다.

환불 요청 5건을 놓고 보면 이렇습니다.

건	금액	요청 내용	사람이 봐야 하나
R1	12,000원	택배 박스가 찢어진 채로 왔어요. 환불해 주세요.	아니오
R2	28,000원	색상이 상세페이지 사진과 너무 달라요. 반품할게요.	아니오

건	금액	요청 내용	사람이 봐야 하나
R3	310,000원	코트를 한 달 입었는데 어깨 솔기가 터졌습니다.	예
R4	45,000원	사이즈 교환하려다가 그냥 환불로 바꿀게요.	예
R5	9,000원	그냥 마음에 안 들어요. 전액 환불해 주세요.	예

R3은 금액이 커서, R5는 금액은 작지만 단순 변심이라 사람이 봐야 합니다. **두 건이 걸리는 이유가 다르다**는 점이 중요합니다. 금액 기준 하나로는 R3만 잡히고 R5는 빠져나갑니다.

기준은 **계산 가능한 형태로** 적어야 에이전트 안의 조건으로 그대로 들어갑니다. "금액이 크면" 이 아니라 "금액이 30,000원을 넘으면", "이상한 요청이면" 이 아니라 "요청에 '단순 변심', '마음에 안' 이 들어 있으면" 처럼 씁니다. 그래야 값을 바꿔 가며 비교할 수 있습니다.

기준을 검증해야 하는 이유

기준을 정했다고 끝이 아닙니다. 그 기준으로 실제로 무엇이 멈추고 무엇이 빠져나가는지 확인해야 합니다. 세 가지를 썹니다.

개입률은 전체 중 사람에게 온 비율입니다. **놓침**은 사람이 봤어야 하는데 자동으로 나가 버린 건이고, 가장 비싼 실수입니다. 사람에게 온 목록에 나타나지 않으니 따로 찾지 않으면 놓쳤다는 사실조차 모릅니다. **헛멈춤**은 볼 필요가 없는데 사람에게 온 건입니다. 헛멈춤이 많으면 담당자가 내용을 보지 않고 승인 버튼만 누르게 됩니다.

아래 셀은 여섯 가지 기준으로 같은 5건을 처리해 세 값을 표로 보여 줍니다. 실행만 하면 됩니다.

실행 되었습니다.

실습: 여섯 가지 멈춤 기준 비교

실행

1

2

3

4

5

6

7

8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36

37

38

39

40

41

42

43

44

45

결과 해석: 기준별 비교표

위에서 아래로 갈수록 기준이 빡빡해지고, **개입률과 놓침이 반대로 움직입니다**. 기준이 없으면 사람은 하나도 보지 않지만 3건을 놓치고, 전부 멈추면 놓침은 없지만 5건을 모두 봅니다. "금액 5천 원 초과" 는 "전부 멈춤" 과 결과가 같습니다. 기준을 너무 낮게 잡으면 기준이 있으나 없으나 같아집니다.

"금액 3만 원 초과 + 표현" 은 놓침과 헛멈춤이 둘 다 0입니다. 금액 기준을 낮추는 대신 **다른 종류의 기준을 하나 더한** 결과입니다. 다만 5건짜리 예제라 0이 나온 것이고, 실제 업무에서는 둘을 동시에 0으로 만드는 기준이 거의 없습니다.

그래서 HITL의 기준은 **정하는 것만큼 검증하는 것이 중요합니다**. 기준 몇 개를 두고 같은 데이터로 세 값을 비교한 뒤, 놓침과 헛멈춤 중 어느 쪽이 더 비싼지를 따져 고릅니다. 프로젝트에서 이 비교표를 직접 만듭니다.

Chapter04

기준이 잘 잡혀도 승인 화면이 나쁘면 HITL은 작동하지 않습니다. 담당자가 판단에 필요한 정보를 못 찾으면 결국 원래 시스템을 따로 열어 확인하게 되고, 그러면 자동화로 아낀 시간이 사라집니다.

목표는 분명합니다. **담당자가 10초 안에 판단할 수 있어야 합니다**.

나쁜 화면과 좋은 화면 비교

같은 건을 두 가지로 보여 주면 차이가 분명합니다.

[나쁜 화면]
요청 ID: R3
판정: 환불 승인
승인하시겠습니까? [승인] [반려]

무엇을 근거로 판단하라는 것인지 알 수 없습니다. 담당자는 주문 시스템을 따로 열어 금액과 구매일을 확인하고, 규정 문서를 찾아 해당 조항을 읽어야 합니다. 이러면 AI가 한 일이 거의 없습니다.

[좋은 화면]
고객 요청: "코트를 한 달 입었는데 어깨 솔기가 터졌습니다."
금액: 310,000원
AI 판정: 환불 승인
근거로 삼은 규정: "제조 불량은 구매 후 1년 이내 전액 환불" (환불규정 3조 2항)
멈춘 이유: 30,000원 초과
통과시키면: 310,000원이 고객 계좌로 환불됩니다
[승인] [금액 수정 후 승인] [반려] [다시 판정]

다섯 가지가 들어 있습니다. **요청 원문, 금액 같은 핵심 정보, AI의 판정과 그 근거가 된 규정 원문, 왜 멈췄는지, 그리고 통과시키면 무슨 일이 일어나는지**입니다.

마지막 줄이 특히 중요합니다. "310,000원이 고객 계좌로 환불됩니다"를 읽고 나면 승인 버튼의 무게가 달라집니다. 무슨 일이 일어나는지 모른 채 누르는 승인이 가장 위험합니다.

네 가지 응답 설계하기

승인과 반려 둘만 두는 경우가 많은데, 실무에서는 이것으로 부족합니다. 가장 흔한 상황이 **"전액은 아니고 절반만"**이나 **"이 문구만 고쳐서"**이기 때문입니다.

둘만 있으면 담당자는 이런 건을 전부 반려한 뒤 직접 처리하게 됩니다. 그러면 자동화가 무너집니다. 네 가지를 둡니다.

승인. 그대로 통과시킵니다.

수정 후 승인. 금액이나 문구를 고쳐서 통과시킵니다. 실무에서 가장 많이 쓰입니다.

반려. 사유와 함께 막습니다. 사유를 함께 받아야 나중에 왜 막았는지 추적할 수 있고, 기준을 고칠 재료가 됩니다.

다시 판정. "부분 환불로 다시 계산해" 처럼 지시를 주고 AI에게 다시 시킵니다.

각 답 뒤에 무엇이 일어나는지도 미리 정해야 합니다. 수정본은 어디에 반영되는지, 반려하면 버리는지 사람에게 넘기는지, 다시 판정에 횟수 상한을 둘지입니다.

대기 건 보관하기

마지막으로 자주 빠뜨리는 것이 있습니다. **승인을 기다리는 시간은 초 단위가 아닙니다.**

담당자는 회의 중일 수도 있고 퇴근했을 수도 있습니다. 31만 원짜리 환불이 내일 아침까지 기다려야 한다면, 그 사이 시스템을 재시작하는 순간 대기 중이던 건이 사라지면 안 됩니다. 고객은 답을 못 받고, 요청이 있었다는 기록조차 남지 않습니다.

그래서 대기 중인 건은 **파일이나 데이터베이스에 저장**해 두어야 합니다. 섹션 2에서 쓴 **InMemorySaver** 는 메모리에만 저장하기 때문입니다. 구조를 짤 때부터 체크포인트를 파일이나 데이터베이스에 두는 쪽으로 정해 두세요.

그리고 운영 규칙을 함께 정해야 합니다. **대기 건이 3일 동안 처리되지 않으면 어떻게 할 것인가.** 자동 승인할지, 자동 반려할지, 다른 담당자에게 넘길지입니다. 정해 두지 않으면 대기 목록이 계속 쌓여 아무도 보지 않는 목록이 됩니다.

Chapter05

프로젝트: 사람이 승인하는 에이전트 만들기

프로젝트에서는 직접 고른 업무에 오늘 살펴본 구조를 붙여, 사람의 승인을 거쳐 결과를 내 보내는 자동화를 만듭니다. 지금까지 만든 자동화에 승인 단계를 더해도 좋고 새로운 업무를 골라도 되지만, 실습에서 다룬 환불 승인은 주제에서 제외합니다.

주제 예시

아래는 주제를 고를 때 참고할 수 있는 예시입니다. 표에 적힌 승인 기준은 하나의 예일 뿐이니, 주제를 고른 뒤에는 실제 업무에 맞게 다시 정하면 됩니다.

자동화	사람의 승인이 필요한 순간	승인 기준 예시
고객 문의 자동 답변	고객에게 답변을 보내기 전	분류 확신도가 0.7 미만이거나 규정 인용이 없는 답변
콘텐츠 자동 발행	구독자에게 발송하기 전	출처가 없는 수치가 들어 있거나 검수 점수가 기준에 못 미치는 글
회의록 할 일 자동 등록	업무 시스템에 등록하기 전	담당자가 불분명하거나 마감이 3일 안에 걸린 항목
재고 자동 발주	거래처에 발주서를 보내기 전	발주액이 100만 원을 넘거나 평소 주문량의 두 배를 넘는 건
문서 자동 수정	원본에 반영하기 전	고친 줄이 20줄을 넘는 수정

자동화	사람의 승인이 필요한 순간	승인 기준 예시
일정 자동 확정	참석자에게 초대 보내기 전	외부 참석자가 포함되거나 기존 일정과 겹치는 건
지원 서류 1차 분류	지원자에게 결과를 알리기 전	불합격으로 분류된 모든 건

필수 구현

1. HITL이 포함된 파이프라인

결과를 바깥으로 내보내는 단계 바로 앞에서 에이전트가 멈추고, 사람의 답을 받은 뒤 멈춘 지점부터 이어서 실행되도록 만듭니다. 승인 기준에 걸리지 않은 건은 사람을 거치지 않고 끝까지 자동으로 처리되어야 하고, 대기 중인 건은 체크포인트에 저장해 두어 담당자가 나중에 확인하더라도 이어서 처리할 수 있어야 합니다.

2. 사람의 승인을 받는 기준과 그 이유

어떤 건에서 사람의 승인을 받을지 계산 가능한 형태로 두 가지 이상 정하고, 각 기준을 왜 두었는지 적습니다. 기준 하나가 어떤 위험을 막기 위한 것인지, 그리고 기준에 걸리지 않아 자동으로 처리되는 건은 왜 사람이 보지 않아도 괜찮은지까지 설명할 수 있어야 합니다.

3. 자동화를 확인할 수 있는 웹 데모

`streamlit` 이나 `gradio` 같은 도구로 사람이 직접 써 볼 수 있는 화면을 만듭니다. 화면에서는 승인을 기다리는 건의 목록을 볼 수 있어야 하고, 한 건을 열었을 때 요청 원문, 에이전트의 판단과 근거, 멈춘 이유, 승인했을 때 일어나는 일을 확인한 뒤 승인이나 반려 같은 답을 줄 수 있어야 합니다. 로컬에서 실행되는 것까지가 필수이고, 공개 배포를 하더라도 API 키는 저장소에 올리지 않습니다.

제출물

실행 파일, 실행 방법을 적은 `README.md`, 그리고 `REPORT.md` 가 들어 있는 깃헙 저장소 주소를 제출합니다.

REPORT.md에 포함할 내용

- 주제와 고른 이유: 어떤 업무를 자동화했고, 어느 단계에서 결과가 바깥으로 나가는지
- 파이프라인 구조도: 노드와 상태(State) 흐름, 멈추는 지점과 재개 경로를 한눈에 볼 수 있는 다이어그램 (Mermaid 코드 또는 이미지)
- 승인 기준과 그 이유: 기준 목록과 각 기준이 막으려는 위험, 자동으로 처리되는 건의 예
- 실행 결과: 몇 건을 처리해 몇 건이 멈췄고, 멈춘 건을 사람이 어떻게 처리했는지

- 데모 화면 설계: 승인 화면에 무엇을 보여 주기로 했고 무엇을 뺐는지, 화면 캡처와 함께
- 프로젝트 회고: 가장 공들인 부분과 앞으로 보완하고 싶은 점